Security Optimization Assessment — Defender for Cloud

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Security Optimization Assessment: Defender for Cloud 4
- What you get . . . 4
- What this delivers . . . 4
- The standard deployment model . . . 4
- Boundary . . . 4
- Effort . . . 5

1. Solution Overview — Security Optimization Assessment: Defender for Cloud 5
- What Defender for Cloud is . . . 5
- Why this assessment matters . . . 5
- The four assessment domains and how they are scored . . . 5
- What is in scope . . . 6
- What is out of scope (and where it lives) . . . 6
- Connections and authentication used . . . 6
- The standard deployment model . . . 7

2. Requirements — Security Optimization Assessment: Defender for Cloud 7
- R1. Azure subscription(s) in scope with Defender for Cloud registered . . . 7
- R2. Read-only roles: Security Reader + Reader at scope . . . 7
- R3. Az PowerShell modules and Resource Graph access . . . 7
- R4. Awareness of the paid-plan landscape (do not assume misconfiguration) . . . 8
- R5. Compliance-standard context . . . 8
- R6. A defined assessment scope and a secure output location . . . 8

Global Requirements — Assessments (category-level) 8
- G1. The workload/tenant being assessed . . . 8
- G2. Read-only assessor access (least privilege) . . . 9
- G3. Microsoft Graph / Az / management API access . . . 9
- G4. Licensing for the products in scope . . . 9
- G5. Scope definition and a point-in-time baseline . . . 9
- G6. Best-practice reference baseline . . . 9
- G7. Output handling and confidentiality . . . 9
- G8. Remediation ownership (for the paired remediation) . . . 10

3. Implementation — Security Optimization Assessment: Defender for Cloud 10
- Stage 1 — Assess (define scope and capture the baseline) . . . 10
- Stage 2 — Configure (set the scoring reference) . . . 11
- Stage 3 — Pilot (score the posture on the pilot subscription) . . . 11
- Stage 4 — Validate (produce and review the findings report) . . . 12

Stage 5 — Expand (broaden scope and trend) . . . 12
5. Testing — Security Optimization Assessment: Defender for Cloud 12
Test 1 — Read-only connection succeeds . . . 12
Test 2 — Discovery captured all four domains . . . 13
Test 3 — Resource Graph coverage query returns . . . 13
Test 4 — Control validation returns evidence, not just a verdict . . . 13
Test 5 — Scoring is reproducible . . . 13
Test 6 — Findings report is prioritized and traceable . . . 13
Test 7 — Drift is detected across runs . . . 13
Test 8 — Output is stored securely . . . 13
Exit criteria . . . 14
6. Troubleshooting — Security Optimization Assessment: Defender for Cloud 14
`Connect-AzAccount` succeeds but the subscription is not visible . . . 14
`Search-AzGraph` throws or returns nothing . . . 14
Secure Score reads 0 or "not available" . . . 14
Plans show Off that you expected On . . . 14
Required compliance standard "not mapped" . . . 14
Coverage query slow or throttled on large estates . . . 15
Scores differ between runs on the same input . . . 15
Findings contradict the live portal . . . 15
7. Best Practices — Security Optimization Assessment: Defender for Cloud 15
Keep it read-only and provably so . . . 15
Score against a named, current baseline . . . 15
Treat unlicensed plans honestly . . . 15
Make the score comparable across runs . . . 16
Allow warm-up before trusting fresh numbers . . . 16
Respect the decision boundary . . . 16
Handle the output as sensitive . . . 16
Where to go next (adjacent catalog areas) . . . 16
8. Deliverables Summary — Security Optimization Assessment: Defender for Cloud 16
Documentation . . . 17
PowerShell scripts (Az + Azure Resource Graph KQL — all read-only) . . . 17
What the buyer receives . . . 18
Outcome . . . 18
Appendix A — PowerShell Scripts 19

Executive Summary — Security Optimization Assessment: Defender for Cloud

White-label · Assessments

A deployed product is not an optimized one. Microsoft Defender for Cloud is frequently switched on for one subscription, left on the free foundational tier, and never tuned — while **Secure Score** stagnates, paid **plans** go unenabled where they matter, the **regulatory-compliance** dashboard is unmapped, and **agent/agentless coverage** has blind spots. This **Security Optimization Assessment** takes a **read-only** look at that configuration, scores it against Microsoft's recommended baseline, and hands you a **prioritized, risk-ranked findings report** telling you what to fix, in what order.

This package is written for an administrator with no prior assessment experience. It is **read-only by design** — every script observes and reports via **Az PowerShell** and **Azure Resource Graph (KQL)**; **none change your environment**. The paired remediation work (closing the gaps) is a separate, pilot-first effort delivered by the **Remediation Playbook for Defender Security Optimization Assessments**.

What you get

- A plain-language **Solution Overview** of Defender for Cloud, the four assessment domains (Secure Score & recommendations, plan coverage, regulatory posture, agent/agentless coverage), and the scoring model.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (category-wide prerequisites live once in the Global Requirements).
- An **Implementation Guide** of exact, ordered steps to run the assessment end to end — enumerate, score, and report — following **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**.
- Five **PowerShell scripts** (Az + Resource Graph KQL) — all **read-only** — for discovery, control validation, posture scoring, an agent/agentless + plan coverage analysis, and the prioritized findings report with drift versus a saved baseline.
- **Testing** and **Troubleshooting** guides so the run is repeatable and trustworthy.

What this delivers

A **scored, evidence-backed picture** of your Defender for Cloud posture: the current Secure Score and top recommendations, which paid plans are on per subscription, how the regulatory-compliance dashboard maps to your standards, and where agent/agentless scanning leaves machines uncovered. The output is a **prioritized remediation backlog** — highest-risk gaps first — ready to hand to the Remediation Playbook.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — assess on a pilot subscription, produce and validate the findings, then expand across subscriptions (and to AWS/GCP connectors) and re-run to trend the score.

Boundary

This assessment supplies **the evidence and the technical score**, not the authorization. **Risk-acceptance** of any finding, **remediation priority** in your context, and any **regulatory / records / incident-response** determination are the customer's security, compliance, and SOC functions' decisions. Plan, pricing, and preview facts are flagged **verify** — confirm current availability in your tenant.

Effort

A competent administrator can run the full assessment on a pilot subscription in a focused engagement, then expand per subscription. It is **repeatable** — re-run on a schedule to trend the posture score.

1. Solution Overview — Security Optimization Assessment: Defender for Cloud

What Defender for Cloud is

Microsoft Defender for Cloud is Azure's **cloud security posture management (CSPM)** and **cloud workload protection (CWPP)** service. For this assessment the relevant capabilities are:

- **Secure Score & recommendations** — a single score summarizing how your resources measure against a security baseline, driven by a prioritized list of **recommendations**.
- **Plan coverage** — Defender for Cloud has a free **foundational CSPM** tier plus paid plans: **Defender CSPM**, and workload plans for **Servers, Storage, SQL/Databases, Containers, App Service, Key Vault, Resource Manager, DNS, APIs**, etc. Each plan unlocks specific protections and assessable posture.
- **Regulatory-compliance dashboard** — maps your posture to standards (Microsoft cloud security benchmark and others you add), showing pass/fail per control.
- **Agent / agentless coverage** — machines are assessed via **agentless machine scanning** (Defender CSPM) and/or agents (Defender for Servers, the Azure Monitor Agent, and integration with Defender for Endpoint). Coverage gaps mean machines are unassessed.

Defender for Cloud also connects to **AWS and GCP** via connectors, extending the same posture model to a multicloud estate.

Why this assessment matters

Most environments have Defender for Cloud **on but not optimized**: Secure Score has plateaued because nobody works the recommendations; paid plans are off on the subscriptions that host sensitive workloads; the compliance dashboard shows a standard nobody chose; and agentless scanning is enabled on one subscription but not the others, so machines silently go unassessed. None of this is obvious from the portal landing page — you have to **enumerate plans, score, recommendations, compliance mappings, and coverage, compare to the recommended baseline, and score it**.

That is what this assessment does. It is **read-only** — it changes nothing — and produces a **prioritized, risk-ranked findings report** across the four domains so remediation goes to the highest-risk gaps first.

The four assessment domains and how they are scored

1. **Secure Score & recommendations** — What is the current score per subscription? Are high-impact and unhealthy recommendations being worked, or is there a large stale backlog?

2. **Plan coverage** — Which paid plans are enabled where? Are the plans that protect the actual workloads (Servers, Storage, SQL, Containers, etc.) on for the subscriptions that host them? *Plan availability/pricing — verify current.*

3. **Regulatory compliance posture** — Is the compliance dashboard enabled and mapped to the standard(s) the organization must meet? What is the pass rate? (Which standard applies is the compliance team's decision.)

4. **Agent / agentless coverage** — Are machines covered by agentless scanning and/or the required agents? Where are the coverage blind spots (unscanned VMs, subscriptions without agentless enabled)?

Each domain is scored against the **best-practice reference baseline** named in the Global Requirements (Microsoft cloud security benchmark / Secure Score guidance — *baselines evolve; verify against current Microsoft guidance*). Domain scores roll up into an overall posture score, and every gap becomes a finding with a **risk rank** (severity × exposure).

What is in scope

• **Reading** Defender for Cloud configuration: plan tiers per subscription, Secure Score and recommendations, compliance assessments, and agent/agentless coverage — via Az and Azure Resource Graph.

• **Scoring** that configuration against the recommended baseline across the four domains.

• **Producing** a prioritized, risk-ranked findings report with a saved baseline for trending.

What is out of scope (and where it lives)

• **Making changes / closing the gaps** — the **Remediation Playbook for Defender Security Optimization Assessments** (pilot-first, `-WhatIf`-gated).

• **First-time enablement and the remediation cadence** — the **Microsoft Defender for Cloud: CSPM & Secure Score Remediation** solution.

• **Cross-pillar (endpoint/identity/email/cloud-apps) suite posture** — the **Security Optimization Assessment — Microsoft Defender (Suite / XDR)** solution.

• **Regulatory, records, incident-response, and risk-acceptance decisions** — the customer's compliance/legal/SOC functions.

Connections and authentication used

• The **Azure portal** → **Microsoft Defender for Cloud** for plans, Secure Score, recommendations, compliance, and coverage (for manual confirmation).

• **Az PowerShell** (`Az.Accounts`, `Az.Security`, `Az.ResourceGraph`, `Az.Resources`) for read-only discovery, validation, scoring, coverage analysis, and reporting; **Azure Resource Graph (KQL)** against `securityresources` for recommendations/coverage.

• Least-privilege roles: **Security Reader** (and **Reader** at the Azure scope) are sufficient for this read-only assessment (per Global Requirements G2).

The standard deployment model

> ***Assess*** *the current Defender for Cloud configuration (plans, Secure Score, recommendations, compliance, coverage) →* ***Configure*** *the assessment scope and baseline reference →* ***Pilot*** *the run on one subscription and review the findings →* ***Validate*** *the score and findings with the security owner →* ***Expand*** *across subscriptions and to AWS/GCP, re-running to trend. Repeatable pattern:* ***Assess*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand****, entirely* ***read-only****.*

2. Requirements — Security Optimization Assessment: Defender for Cloud

The category-wide prerequisites (the target tenant/subscription, read-only assessor access, Graph/Az access, licensing, scope/baseline, output handling, remediation ownership) are documented once in `global-requirements-assessments.md` — read that first. This section lists only the prerequisites **specific to assessing Defender for Cloud**. Each item: **Description / Why required / How to verify / How to implement if missing.**

R1. Azure subscription(s) in scope with Defender for Cloud registered

Description: One or more Azure subscriptions where Defender for Cloud is registered (the `Microsoft.Security` resource provider). **Why required:** Plans, Secure Score, recommendations, and coverage are per-subscription; there is nothing to enumerate without a registered subscription. **How to verify:** `Get-AzContext` shows a subscription; in the portal, **Microsoft Defender for Cloud** loads and shows the subscription under **Environment settings**. **How to implement if missing:** Ensure the subscription exists and `Microsoft.Security` is registered (`Get-AzResourceProvider -ProviderNamespace Microsoft.Security`). Record any unregistered subscription as a **coverage finding**.

R2. Read-only roles: Security Reader + Reader at scope

Description: Security Reader (Defender for Cloud read) and **Reader** at the subscription/management-group scope. **Why required:** The assessment must prove it changes nothing; these read-only roles are sufficient for every script here. **How to verify:** Entra → **Roles and administrators** for Security Reader; Azure → subscription **Access control (IAM)** for Reader. Confirm you can view (not edit) **Environment settings** and recommendations. **How to implement if missing:** Assign **Security Reader** and **Reader**. Do **not** grant Security Admin/Contributor/Owner for the assessment itself.

R3. Az PowerShell modules and Resource Graph access

Description: `Az.Accounts`, `Az.Security`, `Az.ResourceGraph`, `Az.Resources` installed and reachable, with network access to Azure endpoints. **Why required:** The scripts collect plan tiers, Secure Score, recommendations, and coverage through Az and Azure Resource Graph (KQL). **How to verify:** `Connect-AzAccount` succeeds and `SOAMDC-Discovery.ps1` runs; `Search-AzGraph -Query`

`"securityresources | limit 1"` returns without error. **How to implement if missing:** `Install-Module Az -Scope CurrentUser`. Ensure the assessor identity can reach `management.azure.com`.

R4. Awareness of the paid-plan landscape (do not assume misconfiguration)

Description: Understanding that Defender CSPM and the workload plans (Servers, Storage, SQL, Containers, etc.) are **paid** and enabled per subscription. **Why required:** A plan being off is a **coverage finding**, not necessarily a defect — the finding is "enable this plan to gain the protection/posture," which the customer decides. **How to verify:** In **Environment settings** → **Defender plans** per subscription, note which plans are On/Off. *Plan-to-feature mapping and pricing change — verify current.* **How to implement if missing:** Record off-plans as coverage findings with the caveat that enabling them has a cost the customer must accept.

R5. Compliance-standard context

Description: Knowledge of which regulatory standard(s) the organization must meet (so the compliance-domain score is meaningful). **Why required:** The regulatory-compliance posture is scored against the standard the organization actually needs — which standard applies is the compliance team's determination, not the assessment's. **How to verify: Regulatory compliance** in Defender for Cloud shows the enabled standards; confirm with the compliance owner which are required. **How to implement if missing:** If no standard is mapped, record it as a finding ("enable and map the required standard") and note that the *choice* of standard is deferred to compliance/legal.

R6. A defined assessment scope and a secure output location

Description: An agreed scope (which subscriptions/management groups and clouds) and a secure place to store the findings report and the saved baseline. **Why required:** Scope keeps the score comparable across runs; the findings report is sensitive (Global Requirements G5, G7). **How to verify:** The scope is documented; a restricted location exists for `SOAMDC-*.json` outputs and the findings report. **How to implement if missing:** Agree the scope with the security owner before running; create a restricted output location and point the scripts at it.

Global Requirements — Assessments (category-level)

These requirements apply to every solution in the Assessments category (Security Optimization Assessments, remediation playbooks, and cross-cutting security assessments). Published once at the category level so listings stay small. Confirm these before running any assessment. Each entry: Description / Why / Verify / Implement-if-missing.

> *An assessment observes and scores; it does not change your tenant. Every assessment script here is **read-only** (the paired remediation work is separate and pilot-first). Scope, risk-acceptance, and remediation-priority decisions belong to the customer's security/compliance owners — the assessment supplies the evidence, not the authorization.*

G1. The workload/tenant being assessed

Description: An active Microsoft 365 / Entra tenant and/or Azure subscription(s) hosting the products in scope. Why: There is nothing to assess without the target environment. Verify: Sign in to the relevant portal(s) (`security.microsoft.com`, `portal.azure.com`) as an administrator. Implement: Ensure the target tenant/subscription exists and the products under assessment are deployed (even partially — assessing coverage gaps is the point).

G2. Read-only assessor access (least privilege)

Description: A read-only role sufficient to enumerate configuration — e.g. Security Reader, Global Reader, and/or Reader at the Azure scope. Why: An assessment must never require write access; read-only proves it changes nothing. Verify: Entra -> `Roles and administrators`; confirm Security Reader / Global Reader (and Azure Reader where cloud resources are assessed). Implement: Assign the read-only roles; do not grant write/admin roles for the assessment itself.

G3. Microsoft Graph / Az / management API access

Description: Microsoft Graph (Security), Azure Resource Manager, and product-specific management APIs reachable with the assessor identity. Why: The assessment scripts collect configuration and posture via these APIs. Verify: `Connect-MgGraph` (read scopes) and `Connect-AzAccount` succeed; the assessment discovery script runs. *Verify current API/permission scopes.* Implement: Consent the read-only Graph scopes; ensure network access to the Microsoft endpoints.

G4. Licensing for the products in scope

Description: The licenses/plans for the products being assessed (e.g. Defender for Endpoint/Identity/Office 365/Cloud Apps plans, Defender for Cloud plans). Why: Some posture/configuration is only present (and only assessable) when the plan is enabled; the assessment flags plan gaps too. Verify: Confirm the relevant plans in the admin/billing portal. *Verify current plan/feature mapping.* Implement: Note unlicensed products as findings (a coverage gap) rather than skipping them silently.

G5. Scope definition and a point-in-time baseline

Description: A defined assessment scope (which products, subscriptions, and users) and the understanding that results are a point-in-time snapshot. Why: Scope keeps the assessment bounded and the score comparable; posture drifts, so results are dated. Verify: The scope is documented; the assessment records the run date. Implement: Agree the scope before running; re-run periodically to trend the score.

G6. Best-practice reference baseline

Description: The best-practice / benchmark the assessment scores against (Microsoft security baselines, Secure Score, CIS/Zero-Trust as applicable). Why: A score is only meaningful against a reference; the assessment states which baseline it uses. Verify: The reference baseline is named in the findings. *Baselines evolve — verify against current Microsoft guidance.* Implement: Use the documented baseline; note where organizational policy intentionally deviates.

G7. Output handling and confidentiality

Description: A place to store the findings report and an understanding that it may contain sensitive posture detail. Why: Assessment output is a roadmap of your weaknesses; it must be handled as sensitive. Verify: A secure destination for the report exists; access is restricted. Implement: Store findings securely; share on a need-to-know basis. *Risk-acceptance of any finding is the customer's decision.*

G8. Remediation ownership (for the paired remediation)

Description: Named owners for acting on findings (the remediation is separate, pilot-first work). Why: An assessment only creates value when someone owns closing the gaps. Verify: Remediation owners are identified; the Remediation Playbook solution is available for the fixes. Implement: Assign owners; use the Remediation Playbook (pilot-first, -WhatIf-gated) to close findings and re-score.

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation — Security Optimization Assessment: Defender for Cloud

This is the exact, ordered procedure to run the assessment end to end. It is **read-only** — you enumerate configuration, score it, and produce a findings report; you change nothing in the environment.

Naming convention. Every artifact you create uses `<Purpose> – <Scope> – <Stage>`. Use these exact names in the scripts, testing, and the report:

- Assessment scope record → `SOA Scope – Defender for Cloud – Assess`
- Baseline configuration snapshot → `Posture Baseline – Defender for Cloud – Assess`
- Posture score record → `Posture Score – Defender for Cloud – Pilot`
- Prioritized findings report → `Findings Report – Defender for Cloud – Validate`

Placeholders use `contoso.onmicrosoft.com` and a pilot subscription id. Substitute yours. Confirm the prerequisites in Section 2 and the Global Requirements before starting.

Stage 1 — Assess (define scope and capture the baseline)

Step 1.1 — Record the assessment scope

1. Open a working document named `SOA Scope – Defender for Cloud – Assess`.
2. Record: the tenant, the **pilot subscription** and any others in scope, the required compliance standard(s) (per R5), the **run date**, and the **best-practice reference baseline** (Global Requirements G6 — *verify current Microsoft guidance*).
3. Confirm your account has **Security Reader** + **Reader** and the Az modules (Section 2, R2/R3).

Step 1.2 — Sign in and confirm the service

1. Run `Connect-AzAccount` and `Set-AzContext -Subscription '<pilot subscription id>'`.

2. In the portal, open **Microsoft Defender for Cloud** → **Environment settings** and confirm the pilot subscription appears (service registered — R1).

Step 1.3 — Capture the configuration baseline (read-only)

1. Open PowerShell in the `scripts/` folder.

2. Run discovery and save the snapshot as the baseline:

```
.\SOAMDC-Discovery.ps1 -SubscriptionId '<pilot subscription id>' -OutputPath ".\Posture Baseline - Defe
```

1. This enumerates plan tiers, Secure Score, registered providers, Log Analytics workspaces, agentless-scanning state, and any AWS/GCP connectors. It **makes no changes**. Keep the JSON as `Posture Baseline – Defender for Cloud – Assess`.

Stage 2 — Configure (set the scoring reference)

Step 2.1 — Confirm the four assessment domains and weights

1. The scoring model covers: **Secure Score & recommendations**, **plan coverage**, **regulatory compliance posture**, and **agent/agentless coverage**.

2. Review the default weights in `SOAMDC-Score.ps1` (documented in its header). If the security owner wants a different emphasis, record it in `SOA Scope – Defender for Cloud – Assess` so runs stay comparable.

Step 2.2 — Validate that best-practice controls are present (read-only)

1. Run the control validation:

```
.\SOAMDC-Validate.ps1 -BaselinePath ".\Posture Baseline - Defender for Cloud - Assess.json" -RequiredSt
```

1. This checks, per domain, whether the recommended controls exist (Secure Score above a floor with recommendations being worked; the workload plans on for the subscription; the required compliance standard mapped; agentless scanning enabled). Each check returns **Pass / Gap** with evidence — it fixes nothing.

Stage 3 — Pilot (score the posture on the pilot subscription)

Step 3.1 — Analyze plan and agent/agentless coverage

1. Run the coverage analysis (Azure Resource Graph KQL, read-only):

```
.\SOAMDC-Coverage.ps1 -SubscriptionId '<pilot subscription id>' -OutputPath ".\coverage.json"
```

1. This reports which resource types have a matching plan enabled and which machines are/are not covered by agentless scanning — the evidence behind the plan-coverage and agent/agentless domains.

Step 3.2 — Compute the posture score

1. Run the scorer against the validation output:

```
.\SOAMDC-Score.ps1 -ValidationPath ".\SOAMDC-validation.json" -OutputPath ".\Posture Score - Defender f
```

1. This produces per-domain scores and an overall posture score, saved as `Posture Score – Defender for Cloud – Pilot`. Scoring is deterministic. Record the run date.

Stage 4 — Validate (produce and review the findings report)

Step 4.1 — Generate the prioritized findings report

1. Run the report generator (add `-PriorReportPath` on re-runs for drift):

```
.\SOAMDC-Report.ps1 -ValidationPath ".\SOAMDC-validation.json" -ScorePath ".\Posture Score - Defender f
```

1. The report lists every finding with its **domain**, **severity**, **risk rank**, **evidence**, and a **remediation pointer** to the Remediation Playbook, plus **drift** versus any prior baseline.

Step 4.2 — Review with the security owner

1. Walk through `Findings Report – Defender for Cloud – Validate` with the named security owner.

2. The owner confirms **risk-acceptance** and **remediation priority** (their decisions). Record accepted-risk items with justification and expiry. Note that enabling paid plans has a cost the customer accepts.

3. Store the report securely (Global Requirements G7).

Stage 5 — Expand (broaden scope and trend)

Step 5.1 — Widen the scope and re-run

1. Expand `SOA Scope – Defender for Cloud – Assess` to additional subscriptions/management groups and AWS/GCP connectors.

2. Re-run Steps 1.3 → 4.1. With the prior baseline retained, `SOAMDC-Report.ps1` shows the **drift** since the last run.

Step 5.2 — Hand off to remediation and schedule the re-run

1. Hand `Findings Report – Defender for Cloud – Validate` to the **Remediation Playbook** to close the gaps pilot-first.

2. Schedule the assessment on a cadence (for example quarterly) so the Secure Score and coverage trend over time.

Everything above is read-only. If any script attempts a write or requests a write role, stop — it should not.

5. Testing — Security Optimization Assessment: Defender for Cloud

These procedures validate that the assessment **ran correctly and completely** — that the enumeration is trustworthy, the scoring is reproducible, and the findings report is sound. The assessment is read-only, so "testing" proves the *evidence* is reliable. Each test has a **pass/fail gate**. Reference the scripts by the names used in Section 3.

Test 1 — Read-only connection succeeds

Procedure: Run `Connect-AzAccount`, then `.\SOAMDC-Discovery.ps1 -SubscriptionId '<pilot>'`. **Pass:** The script completes and writes `Posture Baseline - Defender for Cloud - Assess.json`; no write role was used. **Fail:** A permissions error, or a prompt for a write role — re-check R2/R3.

Test 2 — Discovery captured all four domains

Procedure: Open the baseline JSON. **Pass:** It contains sections for **Secure Score & recommendations**, **plan coverage**, **regulatory compliance**, and **agent/agentless coverage** — even if a section is empty (an empty section is a finding). **Fail:** A domain section is missing entirely. Re-run; confirm `Microsoft.Security` is registered (R1).

Test 3 — Resource Graph coverage query returns

Procedure: Run `.\SOAMDC-Coverage.ps1 -SubscriptionId '<pilot>'`. **Pass:** The output lists resource types with/without a matching plan and machine agentless-coverage state (or notes "no machines in scope"). **Fail:** `Search-AzGraph` errors — confirm `Az.ResourceGraph` is installed and the Reader role covers the scope (R3).

Test 4 — Control validation returns evidence, not just a verdict

Procedure: Run `.\SOAMDC-Validate.ps1 -BaselinePath ... -RequiredStandard "..."`. **Pass:** Each domain check returns **Pass** or **Gap** with the underlying evidence (e.g. "Secure Score = 42%", "Defender for Servers: Off", "required standard not mapped"). **Fail:** A check returns a verdict with no evidence, or throws.

Test 5 — Scoring is reproducible

Procedure: Run `.\SOAMDC-Score.ps1` twice against the **same** validation output. **Pass:** Both runs produce the same per-domain and overall scores (deterministic). **Fail:** Scores differ for identical input — a defect; do not trust the score until reproducible.

Test 6 — Findings report is prioritized and traceable

Procedure: Run `.\SOAMDC-Report.ps1 ...` and open the report. **Pass:** Findings are sorted by **risk rank**; each names its **domain**, **severity**, **evidence**, and a **remediation pointer** to the Remediation Playbook. **Fail:** Findings are unsorted, or a finding lacks evidence or a remediation pointer.

Test 7 — Drift is detected across runs

Procedure: After a second run, confirm `SOAMDC-Report.ps1` compares to the prior report/baseline. **Pass:** The report shows **new**, **resolved**, and **unchanged** findings (or notes "no prior baseline" on the first run). **Fail:** No drift comparison when a prior baseline exists — check the `-PriorReportPath` argument.

Test 8 — Output is stored securely

Procedure: Confirm the report and score/baseline JSON live in the restricted location agreed in R6. **Pass:** Files are in the secure destination with restricted access. **Fail:** The report is in an open location — move it

(Global Requirements G7).

Exit criteria

All eight tests pass, the security owner has reviewed `Findings Report – Defender for Cloud – Validate`, and risk-acceptance/priority decisions (including paid-plan cost acceptance) are recorded. The backlog is ready for the Remediation Playbook.

6. Troubleshooting — Security Optimization Assessment: Defender for Cloud

Common failures when running the assessment, in **symptom** → **cause** → **fix** form. All fixes stay read-only.

`Connect-AzAccount` succeeds but the subscription is not visible

Symptom: `SOAMDC-Discovery.ps1` reports no subscription, or the wrong one. **Cause:** The default Az context points at a different subscription/tenant. **Fix:** `Set-AzContext -Subscription '<pilot subscription id>'` before running, or pass `-SubscriptionId`. Confirm with `Get-AzContext`.

`Search-AzGraph` throws or returns nothing

Symptom: `SOAMDC-Coverage.ps1` errors on the Resource Graph query, or returns an empty set. **Cause 1:** `Az.ResourceGraph` is not installed. **Cause 2:** The Reader role does not extend to the queried scope. **Fix:** `Install-Module Az.ResourceGraph -Scope CurrentUser`; confirm **Reader** at the subscription/management-group scope (R2/R3). An empty set can also be legitimate (no machines in scope) — the script notes that.

Secure Score reads 0 or "not available"

Symptom: The Secure Score domain shows no score on a subscription you know is active. **Cause:** Defender for Cloud is newly registered and still populating, or `Microsoft.Security` is not registered. **Fix:** Confirm `Get-AzResourceProvider -ProviderNamespace Microsoft.Security` shows Registered. Allow a warm-up window and re-run — Secure Score is not instantaneous (Global Requirements G5).

Plans show Off that you expected On

Symptom: Validation flags Defender for Servers/Storage/SQL as Off. **Cause:** Plans are paid and per-subscription; they are genuinely off, or on at a different subscription than the one in context. **Fix:** Confirm in **Environment settings** → **Defender plans** for the exact subscription. Record off-plans as **coverage findings** (enabling them has a cost — R4).

Required compliance standard "not mapped"

Symptom: The regulatory-compliance domain scores 0 / not-mapped. **Cause:** The standard the organization needs has not been added to the compliance dashboard (or a different standard is mapped).

Fix: Confirm with the compliance owner which standard is required (R5) and pass it as `-RequiredStandard`. If none is mapped, that is a legitimate finding — but the *choice* of standard is the compliance team's, not the assessment's.

Coverage query slow or throttled on large estates

Symptom: `SOAMDC-Coverage.ps1` is slow or returns partial pages. **Cause:** Resource Graph throttling / large result sets across many subscriptions. **Fix:** Scope the run to one subscription/management group at a time and combine baselines; record the scoping in `SOA Scope – Defender for Cloud – Assess`.

Scores differ between runs on the same input

Symptom: `SOAMDC-Score.ps1` gives different numbers for identical validation output. **Cause:** Edited weights, or scoring a fresh discovery rather than the saved validation file. **Fix:** Always pass the **same** validation JSON; confirm the weights in the header were not changed. Scoring is deterministic (Testing Test 5).

Findings contradict the live portal

Symptom: A finding disagrees with what the portal shows now. **Cause:** Posture is a **point-in-time snapshot**; the baseline predates your portal view, or a change was made after the run. **Fix:** Re-run discovery to refresh the baseline, then re-score. Trust the dated baseline over memory (Global Requirements G5).

7. Best Practices — Security Optimization Assessment: Defender for Cloud

Field-proven guidance for running this assessment well and keeping the result trustworthy over time.

Keep it read-only and provably so

- Run with **Security Reader** + **Reader** only. If a script ever attempts a write or requests Security Admin/Contributor, treat it as a defect and stop — this assessment must prove it changed nothing.
- Record the roles and subscription context used alongside the findings report as evidence of read-only execution.

Score against a named, current baseline

- Always record **which** best-practice baseline you scored against and its date (Global Requirements G6). The Microsoft cloud security benchmark and Secure Score controls evolve — *verify against current Microsoft guidance* each cycle.
- Where organizational policy intentionally deviates, mark it as an **accepted deviation** rather than a permanent gap.

Treat unlicensed plans honestly

• A paid plan being off is a **coverage finding**, not automatically a defect. The finding is "enable this plan to gain the protection/posture," and enabling it **has a cost the customer must accept**. Do not present plan-on as free.

• Prioritize plan findings by the **sensitivity of the workloads** on that subscription, not by plan count.

Make the score comparable across runs

• Keep the **scoring weights** fixed between runs (in `SOAMDC-Score.ps1`). If the owner changes a weight, record it in `SOA Scope – Defender for Cloud – Assess` and note the discontinuity.

• Re-use the saved baseline so `SOAMDC-Report.ps1` can show **drift** — a rising Secure Score across runs is the real proof of progress.

Allow warm-up before trusting fresh numbers

• Secure Score, recommendations, and agentless-scan results **take time to populate** after any change. If a subscription was recently modified, note that the snapshot may be mid-warm-up and re-run later.

Respect the decision boundary

• **Risk-acceptance**, **remediation priority**, the **choice of regulatory standard**, and any **incident-response** call belong to the customer's security, compliance, and SOC functions. The assessment supplies evidence and a technical score.

Handle the output as sensitive

• The findings report is a roadmap of your cloud weaknesses. Store it in the restricted location (R6 / Global Requirements G7) and share on a need-to-know basis.

Where to go next (adjacent catalog areas)

• **Remediation Playbook for Defender Security Optimization Assessments** — closes the findings pilot-first, `-WhatIf`-gated, then re-scores. The natural next step.

• **Microsoft Defender for Cloud: CSPM & Secure Score Remediation** — for standing up the remediation cadence and enabling Defender CSPM/agentless scanning if the assessment shows the environment is barely tuned.

• **Security Optimization Assessment — Microsoft Defender (Suite / XDR)** — to assess the whole Defender suite and unified RBAC/incident posture, not just cloud workloads.

• **Security Optimization Assessment — Defender for Cloud Apps** — the sibling assessment for the SaaS/CASB side.

8. Deliverables Summary — Security Optimization Assessment: Defender for Cloud

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box — read-only assessment that scores posture and prioritizes gaps
1. Solution Overview	What Defender for Cloud is, the four assessment domains (Secure Score & recommendations, plan coverage, regulatory posture, agent/agentless coverage), the scoring model, scope, and connections used
2. Requirements Guide	Solution-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Assessments)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps to run the assessment end to end — Assess → Configure → Pilot → Validate → Expand — with every artifact named
5. Testing Guide	Eight validation tests with pass/fail gates proving the evidence is reliable
6. Troubleshooting Guide	Common run failures and fixes
7. Best Practices	Field-proven guidance and where to go next

PowerShell scripts (Az + Azure Resource Graph KQL — all read-only)

Script	What it does
`SOAMDC-Discovery.ps1`	Read-only discovery — Defender for Cloud plan tiers per subscription, Secure Score, registered providers, Log Analytics workspaces, agentless-scanning state, and AWS/GCP connectors; saves the configuration baseline. Makes no changes.
`SOAMDC-Validate.ps1`	Checks, per domain, whether the recommended controls are present (Secure Score above a floor with recommendations worked, workload plans on, required compliance standard mapped, agentless scanning enabled), returning Pass/Gap with evidence.
`SOAMDC-Score.ps1`	Computes deterministic per-domain and overall posture scores against the reference baseline using documented weights.
`SOAMDC-Coverage.ps1`	Read-only Azure Resource Graph (KQL) analysis of plan coverage per resource type and machine agent/agentless coverage — the evidence behind the coverage domains.

Script	What it does
`SOAMDC-Report.ps1`	Produces the prioritized, risk-ranked findings report (domain, severity, evidence, remediation pointer) and shows drift versus a prior saved baseline.

What the buyer receives

- This solution as a **Word** document and **PDF**.
- All five **PowerShell scripts** (read-only, Az + embedded Resource Graph KQL) in a single ZIP.
- A repeatable process following **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — read-only, scored, and traceable, with findings ready to hand to the Remediation Playbook.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A scored, evidence-backed picture of your Defender for Cloud posture across Secure Score & recommendations, plan coverage, regulatory posture, and agent/agentless coverage — and a prioritized remediation backlog that tells you what to fix, in order. Read-only: it changes nothing, it tells you what to fix.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SOAMDC-Coverage.ps1

Purpose: Uses Azure Resource Graph (KQL) against the securityresources / resources tables to produce the coverage evidence behind the plan-coverage and agent/agentless domains: which resource types exist, and Defender for Cloud recommendations/assessments indicating unassessed or uncovered machines. Read-only — runs KQL queries only, changes nothing. HOW TO RUN Connect-AzAccount powershell -ExecutionPolicy Bypass -File .\SOAMDC-Coverage.ps1 ` -SubscriptionId '<pilot subscription id>' -OutputPath ".\coverage.json" Requires Az.ResourceGraph. Read-only roles: Security Reader + Reader. EXPECTED RESULTS A JSON coverage summary: resource-type counts, and machines flagged by coverage-related recommendations. An empty machine set (with a note) is valid when no machines are in scope. .NOTES White-label. Read-only. The KQL below targets Resource Graph tables that evolve — verify current table/column names against Microsoft docs if a query returns unexpected columns. > [CmdletBinding()] param([string]$SubscriptionId, [string]$OutputPath = ".\coverage.json")

How to run: Connect-AzAccount powershell -ExecutionPolicy Bypass -File .\SOAMDC-Coverage.ps1 ` -SubscriptionId '<pilot subscription id>' -OutputPath ".\coverage.json" Requires Az.ResourceGraph. Read-only roles: Security Reader + Reader. EXPECTED RESULTS A JSON coverage summary: resource-type counts, and machines flagged by coverage-related recommendations. An empty machine set (with a note) is valid when no machines are in scope. .NOTES White-label. Read-only. The KQL below targets Resource Graph tables that evolve — verify current table/column names against Microsoft docs if a query returns unexpected columns. > [CmdletBinding()] param([string]$SubscriptionId, [string]$OutputPath = ".\coverage.json")

SOAMDC-Discovery.ps1

Purpose: Read-only discovery for the "Security Optimization Assessment — Defender for Cloud" solution. For the subscription in context (or a named subscription) enumerates: Defender for Cloud pricing/plan tiers (incl. CloudPosture / Defender CSPM and workload plans), current Secure Score, Microsoft.Security provider registration, Log Analytics workspaces, agentless-scanning extension state, and any AWS/GCP security connectors. Writes a JSON baseline (`Posture Baseline - Defender for Cloud - Assess`). Makes NO changes. HOW TO RUN Connect-AzAccount powershell -ExecutionPolicy Bypass -File .\SOAMDC-Discovery.ps1 `-SubscriptionId '<pilot subscription id>'` -OutputPath ".\Posture Baseline - Defender for Cloud - Assess.json" Requires Az.Accounts, Az.Security, Az.Resources (and Az.ResourceGraph for coverage script). Read-only roles: Security Reader + Reader. EXPECTED RESULTS A per-subscription JSON baseline with four domain sections — the evidence the rest of the assessment scores. .NOTES White-label. Read-only. Plan/pricing/preview facts are indicative — verify current availability in your tenant. Secure Score is not instantaneous; allow warm-up after any change. > [CmdletBinding()] param([string]$SubscriptionId, [string]$OutputPath = ".\Posture Baseline - Defender for Cloud - Assess.json")

How to run: Connect-AzAccount powershell -ExecutionPolicy Bypass -File .\SOAMDC-Discovery.ps1 `-SubscriptionId '<pilot subscription id>'` -OutputPath ".\Posture Baseline - Defender for Cloud - Assess.json" Requires Az.Accounts, Az.Security, Az.Resources (and Az.ResourceGraph for coverage script). Read-only roles: Security Reader + Reader. EXPECTED RESULTS A per-subscription JSON baseline with four domain sections — the evidence the rest of the assessment scores. .NOTES

White-label. Read-only. Plan/pricing/preview facts are indicative — verify current availability in your tenant. Secure Score is not instantaneous; allow warm-up after any change. > [CmdletBinding()] param([string]$SubscriptionId, [string]$OutputPath = ".\Posture Baseline - Defender for Cloud - Assess.json")

SOAMDC-Report.ps1

Purpose: Combines the validation output (Pass/Gap per control), the posture score, and the coverage analysis into a single prioritized findings report. Each finding carries: domain, severity, risk rank (severity x exposure), evidence, and a remediation pointer to the Remediation Playbook. If a prior report is supplied, shows new / resolved / unchanged findings (drift). NO changes are made. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SOAMDC-Report.ps1 `-ValidationPath ".\SOAMDC-validation.json"` -ScorePath ".\Posture Score - Defender for Cloud - Pilot.json" `-CoveragePath ".\coverage.json"` -PriorReportPath ".\Findings Report - Defender for Cloud - Validate (previous).json" ` -OutputPath ".\Findings Report - Defender for Cloud - Validate.json" -PriorReportPath is optional (omit on the first run). EXPECTED RESULTS A findings report (JSON + console summary) sorted by risk rank, with drift when a prior run is supplied. This is the backlog handed to the Remediation Playbook. .NOTES White-label. Read-only. Risk-acceptance and remediation priority are the security owner's decisions — this report supplies the evidence and a suggested rank, not the authorization. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ValidationPath, [Parameter(Mandatory)][string]$ScorePath, [string]$CoveragePath, [string]$PriorReportPath, [string]$OutputPath = ".\Findings Report - Defender for Cloud - Validate.json")

How to run: powershell -ExecutionPolicy Bypass -File .\SOAMDC-Report.ps1 `-ValidationPath ".\SOAMDC-validation.json"` -ScorePath ".\Posture Score - Defender for Cloud - Pilot.json" `-CoveragePath ".\coverage.json"` -PriorReportPath ".\Findings Report - Defender for Cloud - Validate (previous).json" ` -OutputPath ".\Findings Report - Defender for Cloud - Validate.json" -PriorReportPath is optional (omit on the first run). EXPECTED RESULTS A findings report (JSON + console summary) sorted by risk rank, with drift when a prior run is supplied. This is the backlog handed to the Remediation Playbook. .NOTES White-label. Read-only. Risk-acceptance and remediation priority are the security owner's decisions — this report supplies the evidence and a suggested rank, not the authorization. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ValidationPath, [Parameter(Mandatory)][string]$ScorePath, [string]$CoveragePath, [string]$PriorReportPath, [string]$OutputPath = ".\Findings Report - Defender for Cloud - Validate.json")

SOAMDC-Score.ps1

Purpose: Consumes the validation output (SOAMDC-Validate.ps1) and computes a posture score per domain and an overall weighted score (0-100). Scoring is DETERMINISTIC for a fixed input so runs are comparable. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SOAMDC-Score.ps1 `-ValidationPath ".\SOAMDC-validation.json"` -OutputPath ".\Posture Score - Defender for Cloud - Pilot.json" SCORING MODEL (default weights — edit here to re-weight; record the change in SOA Scope): secureScore = 25 (the headline posture measure) planCoverage = 30 (protection only exists where the plan is on) regulatoryCompliance = 20 (mapped standard + pass rate) agentCoverage = 25 (unassessed machines are blind spots) Domain score = (passed controls / total controls) * 100. Overall = weighted average. EXPECTED RESULTS A JSON score record: per-domain score, weight, overall score, band, and run date. .NOTES White-label. Read-only. The baseline this scores against evolves — verify current Microsoft guidance each cycle. Weights are a starting point, not a mandate. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ValidationPath, [string]$OutputPath = ".\Posture Score - Defender for Cloud - Pilot.json")

How to run: powershell -ExecutionPolicy Bypass -File .\SOAMDC-Score.ps1 `-ValidationPath ".\SOAMDC-validation.json"` -OutputPath ".\Posture Score - Defender for Cloud - Pilot.json" SCORING MODEL (default weights — edit here to re-weight; record the change in SOA Scope): secureScore = 25 (the headline posture measure) planCoverage = 30 (protection only exists where the plan is on) regulatoryCompliance = 20 (mapped standard + pass rate) agentCoverage = 25 (unassessed machines are blind spots) Domain score = (passed controls / total controls) * 100. Overall = weighted average. EXPECTED RESULTS A JSON score record: per-domain score, weight, overall score, band, and run date. .NOTES White-label. Read-only. The baseline this scores against evolves — verify current Microsoft guidance each cycle. Weights are a starting point, not a mandate. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ValidationPath, [string]$OutputPath = ".\Posture Score - Defender for Cloud - Pilot.json")

SOAMDC-Validate.ps1

Purpose: Consumes the baseline produced by SOAMDC-Discovery.ps1 and evaluates, per domain, whether the recommended controls exist: - Secure Score & recommendations: score above a floor - Plan coverage: the key workload plans on (Servers, Storage, SQL/Databases, Containers, KeyVault) - Regulatory compliance: the REQUIRED standard mapped and enabled - Agent/agentless: agentless scanning enabled and provider registered Each check returns Pass or Gap WITH the evidence. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SOAMDC-Validate.ps1 `-BaselinePath ".\Posture Baseline - Defender for Cloud - Assess.json"` -RequiredStandard "Microsoft cloud security benchmark" `-SecureScoreFloor 70` -OutputPath ".\SOAMDC-validation.json" EXPECTED RESULTS A per-domain Pass/Gap table with evidence — the input to SOAMDC-Score.ps1 and the report. .NOTES White-label. Read-only. Point-in-time snapshot. Plan/preview facts flagged verify. A plan being Off is a coverage finding (enabling it has a cost the customer accepts), not automatically a defect. > [CmdletBinding()] param([Parameter(Mandatory)][string]$BaselinePath, [string]$RequiredStandard = "Microsoft cloud security benchmark", [int]$SecureScoreFloor = 70, [string[]]$RequiredPlans = @('VirtualMachines','StorageAccounts','SqlServers','Containers','KeyVaults'), [string]$OutputPath = ".\SOAMDC-validation.json")

How to run: powershell -ExecutionPolicy Bypass -File .\SOAMDC-Validate.ps1 `-BaselinePath ".\Posture Baseline - Defender for Cloud - Assess.json"` -RequiredStandard "Microsoft cloud security benchmark" `-SecureScoreFloor 70` -OutputPath ".\SOAMDC-validation.json" EXPECTED RESULTS A per-domain Pass/Gap table with evidence — the input to SOAMDC-Score.ps1 and the report. .NOTES White-label. Read-only. Point-in-time snapshot. Plan/preview facts flagged verify. A plan being Off is a coverage finding (enabling it has a cost the customer accepts), not automatically a defect. > [CmdletBinding()] param([Parameter(Mandatory)][string]$BaselinePath, [string]$RequiredStandard = "Microsoft cloud security benchmark", [int]$SecureScoreFloor = 70, [string[]]$RequiredPlans = @('VirtualMachines','StorageAccounts','SqlServers','Containers','KeyVaults'), [string]$OutputPath = ".\SOAMDC-validation.json")